



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 02 - COMO UM LLM “PENSA” E O PAPEL
DO LANGCHAIN4J



OBJETIVOS DA AULA

- ❑ Aprofundar o conceito de Tokens, entendendo diferentes estratégias e suas implicações práticas.
- ❑ Analisar a Janela de Contexto como uma restrição de arquitetura e suas consequências em custo e performance.
- ❑ Compreender Embeddings e a matemática da similaridade de cossenos como base para a busca semântica.
- ❑ Contrastar a chamada direta a uma API de LLM com a abstração oferecida pelo Langchain4j.
- ❑ Mapear os principais componentes do "toolbox" do Langchain4j que serão utilizados no curso.



ANATOMIA DE UMA INTERAÇÃO COM LLM

ANATOMIA DE UMA INTERAÇÃO COM LLM

- ❑ Para um desenvolvedor, um LLM não pode ser uma caixa-preta total. Entender seu fluxo de processamento é crucial para depurar, otimizar e projetar sistemas eficazes.
- ❑ Vamos dissecar os quatro pilares conceituais de uma perspectiva técnica :
 - ❑ Tokenização: A conversão de texto em unidades processáveis.
 - ❑ Janela de Contexto: A limitação fundamental da "memória" do modelo.
 - ❑ Embeddings: A representação vetorial do significado.
 - ❑ Geração Autoregressiva: O processo de construção da resposta.

ENTENDENDO A TOKENIZAÇÃO

- ❑ Por que Tokenizar?
 - ❑ Modelos de machine learning, incluindo LLMs, operam sobre números, não sobre texto cru. A tokenização é o primeiro passo para converter a linguagem humana em um formato numérico que o modelo pode processar.
- ❑ Estratégias de Tokenização:
 - ❑ Word-Based: Divide o texto por espaços. Simples, mas problemático. O vocabulário se torna imenso e não lida bem com palavras raras ou variações (ex: "correr", "correndo", "correu").
 - ❑ Character-Based: Divide o texto em caracteres. Vocabulário pequeno, mas perde muito do significado semântico e gera sequências muito longas.
 - ❑ Subword-Based (O Padrão Moderno): Um meio-termo inteligente. Palavras comuns são um único token, enquanto palavras raras ou complexas são quebradas em sub-unidades de significado. Ex: "imperdoável" -> ["im", "perdo", "ável"]. Algoritmos como Byte-Pair Encoding (BPE), usado pela família GPT, são exemplos dessa abordagem.

IMPLICAÇÕES PRÁTICAS DA TOKENIZAÇÃO

- ❑ Para o desenvolvedor, tokens são uma unidade de medida chave:
 1. Limites da Janela de Contexto: A "memória" do modelo é medida em tokens, não em palavras ou caracteres. Uma frase curta em um idioma pode ter muito mais tokens que em outro.
 2. Custo de API: Provedores de LLM em nuvem cobram por token de entrada e por token de saída. Otimizar a quantidade de tokens em um prompt pode reduzir custos significativamente.
 3. Comportamento Inesperado: Certas tarefas, como contar letras em uma palavra ou inverter uma string, podem falhar porque a representação em tokens não corresponde à nossa intuição. A palavra "strawberry" pode ser tokenizada como ["straw", "berry"], dificultando a contagem de 'r's.

APROFUNDANDO NA JANELA DE CONTEXTO

- ❑ É a "Memória RAM" da Requisição
- ❑ A janela de contexto é a quantidade máxima de tokens que o modelo pode considerar em uma única chamada (prompt de entrada + resposta gerada). É uma limitação arquitetural fundamental.
- ❑ O Custo Quadrático da Atenção: O mecanismo central dos modelos Transformer é a auto-atenção, onde cada token "presta atenção" a todos os outros tokens na janela para entender o contexto. A complexidade computacional e de memória desse mecanismo é quadrática em relação ao tamanho da sequência de tokens ($O(n^2)$).
- ❑ Implicações: Dobrar a janela de contexto quadruplica a carga computacional. É por isso que janelas maiores são mais caras e lentas. Quando uma conversa excede a janela, as informações mais antigas são descartadas (truncadas), e o modelo "esquece" o início do diálogo.

EMBEDDINGS E SIMILARIDADE

- ❑ Embeddings usando geometria para traduzir significados
- ❑ Um embedding é um vetor denso de números de ponto flutuante que representa o significado semântico de um token ou de um trecho de texto em um espaço vetorial de alta dimensão.
- ❑ A Métrica Chave: Similaridade de Cossenos (Cosine Similarity)
- ❑ Para comparar o quão "próximos" em significado dois textos são, usamos a similaridade de cossenos, que mede o cosseno do ângulo entre seus vetores de embedding.
 - ❑ Resultado 1: Vetores apontam na mesma direção (ângulo de 0°). Significado idêntico.
 - ❑ Resultado 0: Vetores são ortogonais (ângulo de 90°). Sem relação semântica.
 - ❑ Resultado -1: Vetores apontam em direções opostas (ângulo de 180°). Significados opostos.
- ❑ Por que isso é fundamental? Este é o mecanismo por trás da etapa de Retrieval no RAG. Para encontrar documentos relevantes para uma pergunta, o sistema calcula o embedding da pergunta e busca no banco de dados vetorial os embeddings de documentos com a maior similaridade de cossenos.

LANGCHAIN4J: A SOLUÇÃO PARA O DESACOPLAMENTO DE APIS

ACOPLAMENTO COM A API DO PROVEDOR

- ❑ Cada provedor de LLM expõe uma API REST distinta. Para interagir com nosso modelo `gpt-oss:20b` rodando no Ollama, teríamos que fazer uma chamada HTTP POST para o endpoint `/api/chat`.

- ❑ Exemplo de Chamada Direta (usando `curl`):

```
curl http://localhost:11434/api/chat -d '{
  "model": "gpt-oss:20b",
  "messages": [
    {
      "role": "user",
      "content": "Qual a melhor época para visitar o Japão?"
    }
  ],
  "stream": false
}'
```

- ❑ Problema: Se amanhã quisermos testar um modelo da OpenAI ou do Google, teríamos que reescrever todo o nosso código de cliente HTTP para se adequar a um novo formato de JSON, novos cabeçalhos de autenticação e novos endpoints. Isso gera um forte acoplamento e dificulta a manutenção e evolução do sistema.

LANGCHAIN4J COMO ABSTRAÇÃO

- ❑ O Padrão "JDBC para IA"
 - ❑ Langchain4j resolve esse problema fornecendo uma interface unificada que abstrai os detalhes de cada provedor.
- ❑ A Vida Com Langchain4j:
 - ❑ Nós interagimos com interfaces de alto nível, como ChatLanguageModel.
 - ❑ A implementação específica (Ollama, OpenAI, etc.) é injetada em tempo de execução com base em uma simples configuração.
 - ❑ Código da Aplicação (Permanece o mesmo):

```
// Interface declarativa. A implementação é gerenciada pelo framework.
```

```
@RegisterAiService  
  
interface Assistant {  
    String chat(String userMessage);  
}
```

```
// Injeção e uso no nosso serviço de negócio.
```

```
@Inject  
Assistant assistant;  
  
public String ask(String question) {  
    return assistant.chat(question);  
}
```

LANGCHAIN4J COMO ABSTRAÇÃO

- ❑ Configuração (application.properties):

```
# Para usar o Ollama local
```

```
quarkus.langchain4j.ollama.chat-model.model-id=gpt-oss:20b
```

```
# Para mudar para OpenAI, basta comentar a linha acima e descomentar as abaixo
```

```
# quarkus.langchain4j.openai.chat-model.model-name=gpt-4o
```

```
# quarkus.langchain4j.openai.api-key=sk-...
```

- ❑ Essa abordagem desacopla nossa lógica de negócio da implementação do provedor de IA, evitando o vendor lock-in e permitindo uma flexibilidade estratégica imensa.



O "TOOLBOX" DO LANGCHAIN4J

- ❑ Langchain4j não é apenas um conector. Ele fornece um conjunto de componentes (um "toolbox") para construir aplicações de IA complexas, implementando padrões de design comuns.
- ❑ Principais Componentes que Usaremos:
 - ❑ `ChatLanguageModel`: A abstração central para interagir com modelos de chat como o `gpt-oss:20b`.
 - ❑ `EmbeddingModel`: Interface para converter texto em vetores (embeddings).
 - ❑ `EmbeddingStore`: Abstração para bancos de dados vetoriais (onde os embeddings são armazenados e consultados).
 - ❑ `DocumentSplitter`: Utilitários para quebrar documentos grandes em pedaços (chunks) otimizados para RAG.
 - ❑ `RetrievalAugmentor`: O componente que orquestra o pipeline RAG (busca no `EmbeddingStore` e aumenta o prompt).
 - ❑ `@Tool`: Anotação para expor métodos Java como "ferramentas" que um agente de IA pode invocar.
 - ❑ `ChatMemory`: Gerencia o histórico da conversação para manter o contexto entre as interações.

**PRÓXIMOS PASSOS: MÃO NA
MASSA!**

PRÓXIMOS PASSOS: MÃO NA MASSA!

❑ Na próxima aula nós vamos:

1. Criar nosso projeto Quarkus com as dependências do Langchain4j e Ollama.
2. Configurar o application.properties para conectar ao nosso modelo gpt-oss:20b.
3. Implementar nosso primeiro AiService declarativo.
4. Construir um endpoint JAX-RS para expor nosso serviço e fazer a primeira chamada de IA, validando todo o ambiente.

ATÉ A PRÓXIMA!